

Buffer Sequence Adaptors

Contents

1. [Overview](#)
2. [Motivation and Scope](#)
3. [Impact On the Standard](#)
4. [Design Decisions](#)
5. [Proposed Wording](#)
6. [Acknowledgements](#)

1. Overview

This document proposes additional algorithms and types for working with the buffer sequence concepts defined in [networking.ts].

2. Motivation and Scope

[networking.ts] defines a buffer sequence as a bidirectional range of non-owning references to individual contiguous memory regions. The adaptors and types proposed in this paper allow for simply lazy transformations of buffer sequences to achieve effects commonly used in implementations which operate on buffers.

2.1 Target Audience

The algorithms and types provided here are aimed squarely at developers working with [networking.ts].

2.2 Reference Implementation

The Boost.Beast library, from portions of which this proposal is derived, has been deployed in a number of production systems, such as internet-facing HTTP servers, decentralized cryptocurrency networks, and finance applications. The Boost.Beast library has been used on the following platforms:

- Win32 using Visual Studio 14 and 15
- Linux (newer kernels) using g++ 4.8 or later
- Mac OS X 10.4 using clang 3.6 or later

2.3 Related Work

The interfaces used for transacting with buffer sequences are based on idioms established in [networking.ts]. library.

3. Impact On the Standard

This is a pure library proposal. It does not add any new language features, nor does it alter any existing standard library headers. This library can be implemented using compilers that conform to the C++17 standard. However, the library also requires [networking.ts].

4. Design Decisions

The prefix buffers_ was chosen because buffer sequences can contain multiple memory regions. This distinguishes the algorithms from those functions which operate on types meeting the requirements of *DynamicBuffer*, which use the prefix buffer_ (these algorithms are present in Boost.Beast but not proposed here).

The types `buffers_cat_view` and `buffers_prefix_view` are made public so that callers can build abstractions which refer to the type. The ranges produced by the library are simple adaptors over existing memory buffers with reference semantics. Memory allocations are not required in the implementation.

5. Proposed Wording (informative)

Exact wording is to be determined after [networking.ts] is merged

5.1 Definitions

1. The following definitions are adopted from [networking.ts] as-is:

- Types `const_buffer` and `mutable_buffer`.
- Concepts *ConstBufferSequence* and *MutableBufferSequence*.
- The functions `buffer_size`, `buffer_sequence_begin`, and `buffer_sequence_end`.
- Class templates `is_const_buffer_sequence_v` and `is_mutable_buffer_sequence_v`.

2. Define `BUFFER_TYPE(TN...)` as follows:

- The type `mutable_buffer` if `(... && is_mutable_buffer_sequence_v<TN>)` is true, otherwise
- The type `const_buffer`.

[Note] possible implementation

```
template <class... TN>
using BUFFER_TYPE = conditional_t<
    (... && is_mutable_buffer_sequence_v<TN>), mutable_buffer, const_buffer>;
```

5.2 Header `<experimental/buffer>` additions

```
namespace std {
namespace experimental {
namespace net {

template <class... BN>
class buffers_cat_view;

template <class BS>
class buffers_prefix_view;

template <class BS>
class buffers_suffix;

template <class... BN>
auto buffers_cat(BN const&... buffers) -> buffers_cat_view <BN...>;

template <class BS>
auto buffers_front (BS const& buffers) -> see-below;

auto buffers_prefix (size_t size, const_buffer buffer) -> const_buffer;
auto buffers_prefix (size_t size, mutable_buffer buffer) -> mutable_buffer;

template <class BS>
auto buffers_prefix (size_t size, BS const& buffers) -> buffers_prefix_view <BS>;

} // net
} // experimental
} // std
```

5.3 Class template `buffers_cat_view`

This class represents the buffer sequence created when a variable number of existing buffer sequences of possibly dissimilar types are concatenated in order, with empty buffers in the underlying sequences removed when at least one

sequence is present.

```
namespace std {
namespace experimental {
namespace net {

template <class... BN>
class buffers_cat_view
{
    tuple <BN...> bn_; // exposition only

public:
    using value_type = see-below;

    using const_iterator = see-below;

    explicit buffers_cat_view (BN const&... buffers);

    buffers_cat_view (buffers_cat_view const& other);

    const_iterator begin() const noexcept;
    const_iterator end() const noexcept;
};

} // net
} // experimental
} // std
```

1. using value_type = see-below

Type: *BUFFER_TYPE*(BN...)

2. Class template `buffers_cat_view` satisfies *ConstBufferSequence*. If `value_type` is *mutable_buffer*, then `buffers_cat_view` also satisfies *MutableBufferSequence*. Template argument `BN` requires that `sizeof...(BN) > 0` and that `(... && is_const_buffer_sequence_v<BN>)` is true.

3. using const_iterator = see-below

An iterator which satisfies all the requirements for bidirectional iterators (C++ 2014 [bidirectional.iterators]) except that:

- The reference type is the same as `buffers_cat_view::value_type`, and
- There is no requirement to provide `operator->`.

4. explicit buffers_cat_view (BN const&... buffers)

Effects: Constructs a buffer sequence which when iterated represents the concatenation of the sequences in `buffers`, except that memory regions of size zero are not visited. The implementation shall maintain a copy of each value in `buffers` until the view is destroyed

5. buffers_cat_view (buffers_cat_view const& other)

Effects: Constructs a copy of `other`.

6. const_iterator begin() const noexcept

Returns: An iterator pointing to first buffer in the view which does not have a zero size, otherwise returns `this->end()`.

7. const_iterator end() const noexcept

Returns: An iterator pointing to one past the last buffer in the view which does not have a zero size.

5.4 Class template `buffers_prefix_view`

This class represents the buffer sequence created by forming the prefix of an existing buffer sequence.

```
template <class BS>
class buffers_prefix_view
{
    BS bs_; // exposition only

public:
    using value_type = see-below;

    using const_iterator = see-below;

    buffers_prefix_view (size_t size, BS const& buffers);

    template <class... Args>
    buffers_prefix_view (size_t size, in_place_t, Args&&... args);

    buffers_prefix_view (buffers_prefix_view const& other);

    const_iterator begin() const noexcept;
    const_iterator end() const noexcept;
};
```

1. using value_type = see-below

Type: *BUFFER_TYPE*(BS)

2. Class template `buffers_prefix_view` satisfies *ConstBufferSequence*. If `value_type` is `mutable_buffer`, then `buffers_prefix_view` also satisfies *MutableBufferSequence*. Template argument `BS` satisfies *ConstBufferSequence*

3. using const_iterator = see-below

An iterator which satisfies all the requirements for bidirectional iterators (C++ 2014 [bidirectional.iterators]) except that:

- The reference type is the same as `buffers_prefix_view::value_type`, and
- There is no requirement to provide operator `->`.

4. buffers_prefix_view (size_t size, BS const& buffers)

Effects: Constructs a buffer sequence which when iterated represents a prefix of buffers with size `min(size, buffer_size(buffers))`. The implementation shall maintain a copy of `buffers` until the view is destroyed.

5. template <class... Args>
buffers_prefix_view (size_t size, in_place_t, Args&&... args)

Effects: Constructs a buffer sequence which when iterated, represents a prefix of the buffer sequence `b` formed by `BS(forward<Args>(Args)...)...`, with size `min(size, buffer_size(buffers))`. The implementation shall maintain a copy of `b` until the view is destroyed.

6. buffers_prefix_view (buffers_prefix_view const& other)

Effects: Constructs a copy of `other`.

7. const_iterator begin() const noexcept

Returns: An iterator pointing to the beginning of the buffer sequence.

8. const_iterator end() const noexcept

Returns: An iterator pointing to one past the end of the buffer sequence.

5.5 Class `buffers_suffix`

This class template represents the buffer sequence created by forming a suffix of an existing buffer sequence. A member function allows the caller to decrease the length of the suffix by a specified amount.

```
template <class BS>
class buffers_suffix
{
    BS bs_; // exposition only

public:
    using value_type = see-below;

    using const_iterator = see-below;

    explicit buffers_suffix (BS const& buffers);

    template <class... Args>
    buffers_suffix (in_place_t, Args&&... args);

    buffers_suffix (buffers_suffix const& other);

    void consume (size_t amount) noexcept;

    const_iterator begin() const noexcept;

    const_iterator end() const noexcept;
};
```

1. using value_type = *see-below*

Type: *BUFFER_TYPE*(BS)

2. Class template buffers_suffix satisfies *ConstBufferSequence*. If value_type is mutable_buffer, then buffers_suffix also satisfies *MutableBufferSequence*. Template argument BS satisfies *ConstBufferSequence*

3. using const_iterator = *see-below*

An iterator which satisfies all the requirements for bidirectional iterators (C++ 2014 [bidirectional.iterators]) except that:

- The reference type is the same as buffers_suffix::value_type, and
- There is no requirement to provide operator->.

4. buffers_suffix (BS const& buffers)

Effects: Constructs a buffer sequence which when iterated represents a suffix of buffers with size equal to buffer_size(buffers). The implementation shall maintain a copy of buffers until the buffer sequence is destroyed.

5. template <class... Args>
buffers_suffix (size_t size, in_place_t, Args&&... args)

Effects: Constructs a buffer sequence which when iterated, represents a suffix of the buffer sequence b formed by BS(forward<Args>(Args)...), with size equal to buffer_size(b). The implementation shall maintain a copy of b until the view is destroyed.

6. buffers_suffix (buffers_suffix const& other)

Effects: Constructs a copy of other.

7. void consume (size_t amount) noexcept

Effects: Reduces the size of the suffix by amount. If amount is greater than or equal to the current size of the suffix, the resulting size of the suffix will become zero.

8. const_iterator begin() const noexcept

Returns: An iterator pointing to the beginning of the buffer sequence.

9. `const_iterator end() const noexcept`

Returns: An iterator pointing to one past the end of the buffer sequence.

5.6 Function `buffers_cat`

This function returns a view representing a buffer sequence which, when iterated, produces the concatenation of the passed buffer sequences.

```
template <class... BN>
auto buffers_cat(BN const&... buffers) -> buffers_cat_view <BN...>;
```

1. *Requires:* `sizeof...(BN) > 0` and `(... && is_const_buffer_sequence_v<BN>)` is true.

2. *Returns:* `buffers_cat_view<BN>(buffers)`.

5.7 Function `buffers_prefix`

This function returns a buffer sequence representing a prefix of the passed buffer sequence.

```
auto buffers_prefix (size_t size, const_buffer buffer) -> const_buffer;

auto buffers_prefix (size_t size, mutable_buffer buffer) -> mutable_buffer;

template <class BS>
auto buffers_prefix (size_t size, BS const& buffers) -> buffers_prefix_view <BS>;
```

1. `const_buffer buffers_prefix (size_t size, const_buffer buffer)`

Returns: `const_buffer (buffer.data(), min(size, buffer.size()))`.

2. `mutable_buffer buffers_prefix (size_t size, mutable_buffer buffer)`

Returns: `mutable_buffer (buffer.data(), min(size, buffer.size()))`.

3.

```
template <class BS>
buffers_prefix_view <BS>
buffers_prefix (size_t size, BS const& buffers);
```

Requires: BS satisfies *ConstBufferSequence*.

Returns: `buffers_prefix_view (buffers)`.

Note: This function participates in overload resolution only when:

- `is_same_v<BS, const_buffer>` is false,
- `is_same_v<BS, mutable_buffer>` is false, and
- `is_const_buffer_sequence_v<BS>` is true.

5.8 Function `buffers_front`

This function returns the first buffer in a buffer sequence, or an empty buffer if the buffer sequence is an empty range.

```
template <class BS>
auto buffers_front (BS const& buffers) -> see-below;
```

1. *Requires:* BS satisfies *ConstBufferSequence*.

2. *Type:* The return type is `decltype(buffer_sequence_begin(buffers))::const_iterator::value_type`.

3. *Returns:* If `buffer_sequence_begin(buffers) != buffer_sequence_end(buffers)` then the return value is constructed from `*buffer_sequence_begin(buffers)`, otherwise the return value is default-

constructed.

Acknowledgements

Many thanks to Agustín Bergé for showing me the ropes with some of the wording.